Algorithm: Support Vector Machine (SVM)

Dataset: Breast Cancer Dataset (from sklearn.datasets)

- ***"Created a new notebook with custom implementation"***

Task	**Actions Taken**	**Rationale**	**Difficulty Level (1-10)**	**Time Taken**
Data Understanding	Loaded and explored the Breast Cancer dataset, examined features, and identified target labels.	To understand the dataset structure and ensure it is suitable for classification.	3	20 min
Data Preprocessing	Split data into 75% training and 25% testing, applied feature scaling using StandardScaler, and transformed labels to -1 and 1.	Feature scaling improves model convergence, and label transformation aligns with SVM implementation.	5	30 min
Implemented Custom SVM	Coded Support Vector Machine from scratch, used a polynomial kernel (degree=3), optimized with mini-batch SGD.	Understanding SVM internals and kernel transformations while optimizing performance.	8	2 hours
Implemented Sklearn SVM	Used SVC from Scikit-learn with an RBF kernel for performance comparison.	To validate correctness of the custom implementation and compare performance.	3	30 min
Hyperparameter Tuning	Tuned learning rate (0.001), regularization parameter (λ=0.01), mini-batch size (10), and iterations (5000).	Finding optimal parameters ensures stable convergence and good generalization.	6	50 min

Model Evaluation	Computed accuracy for both custom and Sklearn implementations (0.9860) and analyzed differences.	To assess effectiveness and confirm the accuracy of the custom model.	4	30 min
Conclusion & Future Improvements	Summarized results, noted matching accuracy with Sklearn, and planned further kernel experiments.	To document findings and explore enhancements for future improvements.	3	20 min

1. Business Understanding:

Objective of this study is to develop a Support Vector Machine (SVM) classifier from scratch and compare results with Scikit-learn's own implementation of the SVM classifier, all while trying to classify breast cancer tumors as malignant or benign based on various features input into the model. The main purpose would thus encompass l**earning more on SVMs**, **hyperparameter tuning** (e.g., **the learning rate, lambda regularization, and kernel functions**), and model evaluation with the **accuracy metric**.

The challenges will be ensuring that the numerical stability has been trained well enough, the correct kernel implementation and the efficient optimization of the SVM loss function.

2. Data Understanding

- The Dataset Used: **`load_breast_cancer()`** from Scikit-Learn
- Feature Description:
    - It has about **30 numerical features**, which are the different characteristics that the cell nuclei possess (e.g., **radius, texture, smoothness, compactness**).
- Target variable is binary, **`0` (malignant) or `1` (benign).**
- Initial Observations:
    - The data is ready for classification.
    - It requires feature standardization as the scale of features varies.

3. Data Preparation

- Changes Made: divided into **75% training and 25% testing**
- Feature scaling made with **`StandardScaler`** for normalized data for better convergence of the model.

- Transformed class labels from **`0` and `1` to `-1`** (malignant) and `1` (benign) to match the custom SVM's format.

4. Modelling

4.1 Custom Implementation of SVM

Implementation Details:

- Feature mapping was accomplished with a polynomial transformation kernel (**degree = 3**).
- Optimized using mini-batch Stochastic Gradient Descent (**mini-batch = 10**).
- **Learning Rate = 0.001**, Regularization parameter **(lambda) = 0.01**.
- Further Number of iterations = **5000**.
- Weights randomized.
- Decision function: **`sign(w . X + b)`**
- Issues with Identified:
- Initial misalignment of shape issues during implementation was subsequently solved using the **transformation of kernel**.
- Improved training stability with appropriate selection of learning rate.

5. Evaluation

Implementation	Accuracy
Custom SVM (Poly)	0.9860
Scikit-learn SVM (RBF)	0.9860

Observations:

Both models are accurate because their outputs are just as close to each other as those from other models, proving the custom implementation is working perfectly. Observed performance change with different kernels or data sets.

Summary:

Successfully created an SVM classifier from scratch using a **polynomial kernel**. **Feature scaling and mini-batch gradient descent** were applied for stable training. The accuracy achieved was identical (**0.9860**) as compared to that of Scikit-learn's SVM with an **RBF kernel**.